

Refactorización de Código con Clases y Principios SOLID

Objetivo de la actividad

El objetivo de esta práctica es transformar una aplicación web interactiva que está escrita de forma puramente funcional/estructurada hacia un diseño limpio, escalable y mantenible utilizando Clases en JavaScript y el Principio de Responsabilidad Única (SRP) de SOLID.

Contexto del problema

Dispones de una aplicación HTML/JS que calcula la edad equivalente de una mascota en años humanos (multiplicando la edad real por un factor de 7). Actualmente, todo el código está mezclado en un único script: la lógica matemática, la lectura del DOM, la manipulación de estilos y los eventos se ejecutan de forma secuencial.

Tu misión es refactorizar este código para que el software sea modular y fácil de ampliar en el futuro (por ejemplo, si mañana quisiéramos cambiar la interfaz visual o añadir diferentes tipos de mascotas).

Requisitos de la Refactorización

Para cumplir con el principio SRP (Single Responsibility Principle), debes dividir el programa en tres clases independientes, cada una con una única razón para cambiar:

1. Clase AgeCalculator (Lógica de Negocio)

Debe encargarse exclusivamente de las reglas matemáticas y validaciones del cálculo, sin saber nada de la pantalla o del HTML.

- Debe aceptar un factor de conversión en su constructor (por defecto igual a 7).
- Debe tener un método isValid(age) que devuelva si la edad es un número válido (mayor que 0).
- Debe tener un método convertToHumanYears(age) que realice la operación matemática.

2. Clase DOMManager (Interfaz de Usuario)

Debe encargarse exclusivamente de interactuar con el HTML. No sabe cómo se calcula la edad, solo sabe leer y pintar en la pantalla.

- En su constructor, debe seleccionar y almacenar los elementos del DOM necesarios (#edad-mascota, #btn-calcular, #resultado-mascota).
- Debe tener un método getInputValue() que devuelva el número introducido por el usuario.
- Debe tener un método renderMessage(text, color) que pinte el resultado en el contenedor correspondiente con el color indicado (rojo para errores, azul para éxito).
- Debe tener un método bindCalculateEvent(handler) que escuche el evento click del botón y ejecute una función cuando ocurra.

3. Clase AppController (Orquestador / Controlador)

Es el "cerebro" que conecta las dos clases anteriores. Recibe las instrucciones de la vista (DOM) y le pide los cálculos al modelo (Calculator).

- Su constructor debe recibir una instancia de AgeCalculator y otra de DOMManager.
- Debe tener un método init() para activar los eventos de la aplicación.
- Debe tener un método privado o interno (ej. handleCalculation()) que recoja la edad del DOM, valide si es correcta, calcule el resultado si es válida, y ordene al DOM pintar el mensaje final o el error.

Instrucciones para la entrega

1. Crea un archivo HTML con la estructura base del formulario.
2. Escribe las tres clases dentro de la etiqueta <script>.
3. Instancia las clases al final del script y arranca la aplicación llamando al método de inicialización del controlador.
4. Pregunta de reflexión: Si mañana quisiéramos que la calculadora use el factor de un gato (factor = 5) en lugar de un perro (factor = 7), ¿qué línea de código tendrías que modificar y por qué este diseño hace que sea tan fácil?